Tutoriel : création d'une application iOS avec deux View Controllers dont le second est un `TableViewController` avec des switchs

Cette fois encore nous allons créer une application iOS avec deux écrans et un passage d'information du premier au second, mais cette fois le second sera directement un TableViewController qui sera agrémenté de switchs pouvant signifier la présence d'un étudiant.

Création du projet Xcode

Comme précédemment, nous devons créer un projet Xcode. Cette fois nous l'appellerons `TableViewWithChecks Tuto`.

Création de l'interface

Sélectionnez le `Main.storyboard` à gauche, la silhouette d'un iPhone apparaît. Cliquez ensuite sur le + en à gauche de l'écran pour faire apparaitre la bibliothèque de *widgets* que vous pouvez ajouter. Tapez `controller` dans le champ `Objects` pour filtrer la recherche.

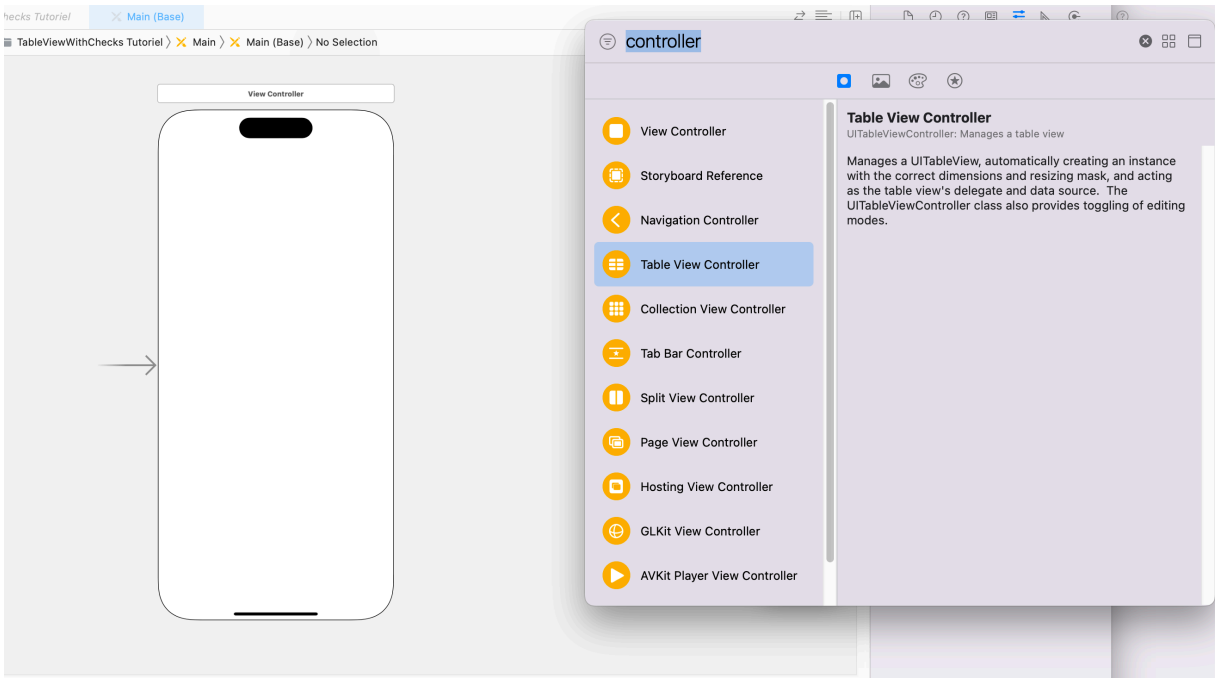


Figure 1Sélectionnez Table View Controller.

Sélectionnez cette fois (c'est plus rapide pour cet exemple) `Table View Controller` et faites-le glisser à côté du `ViewController` actuel.

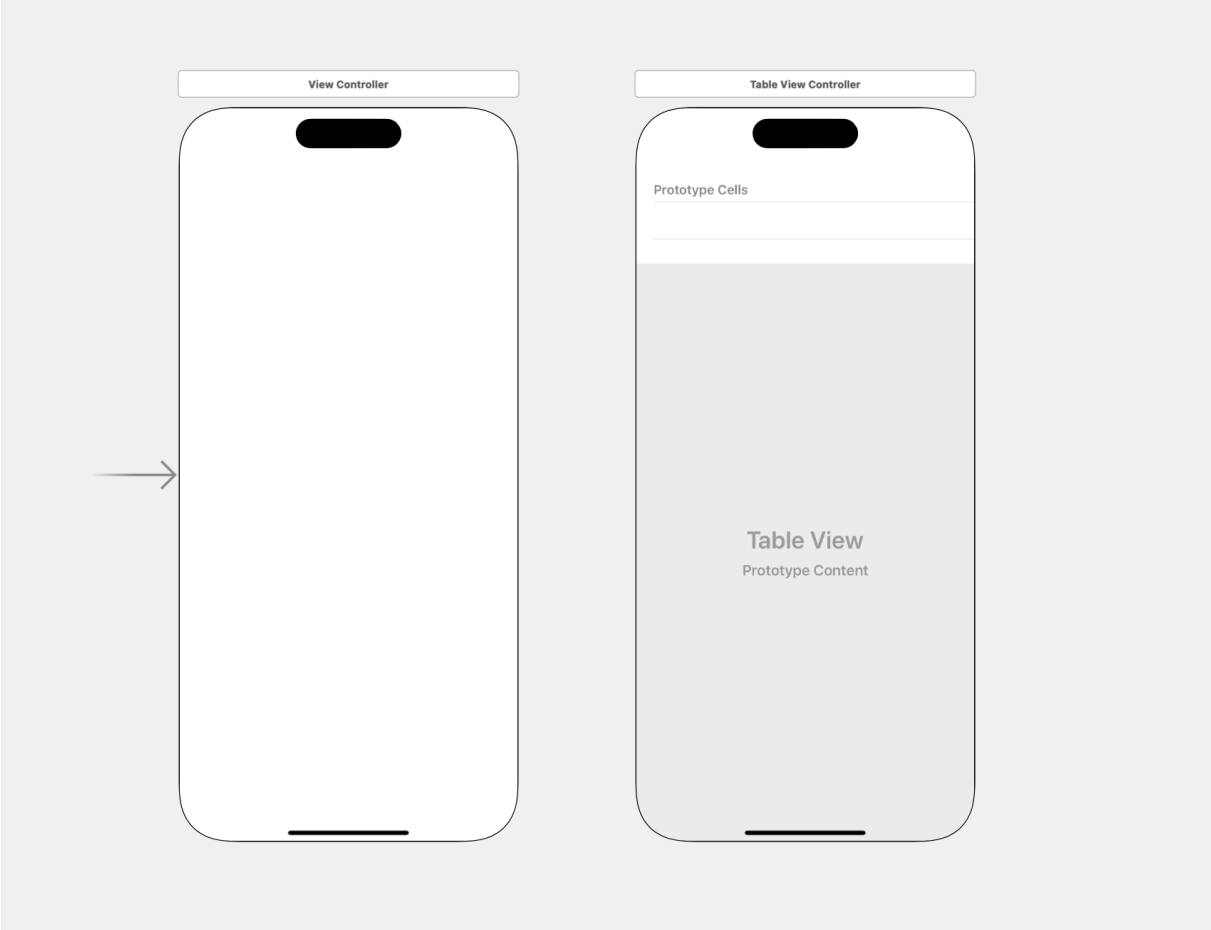
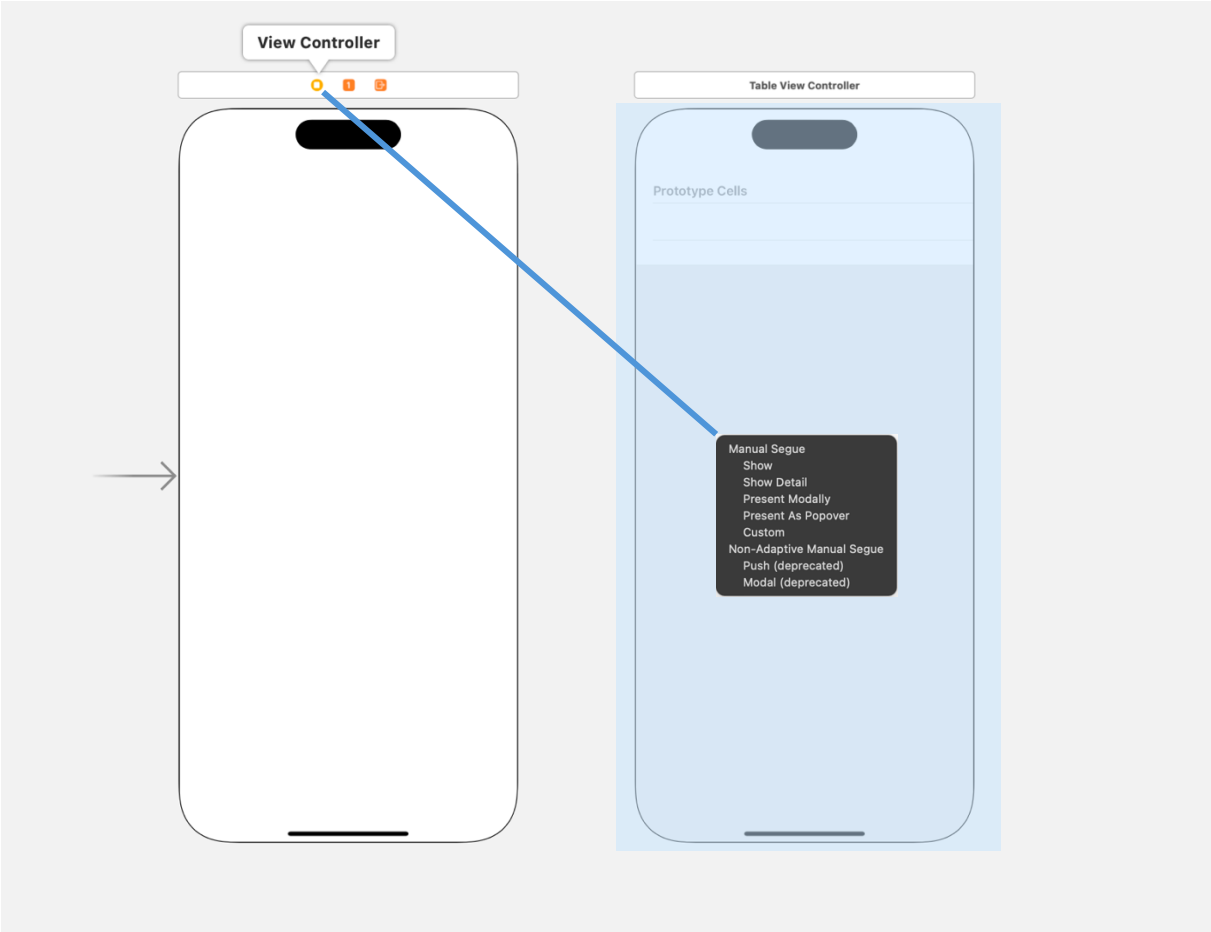
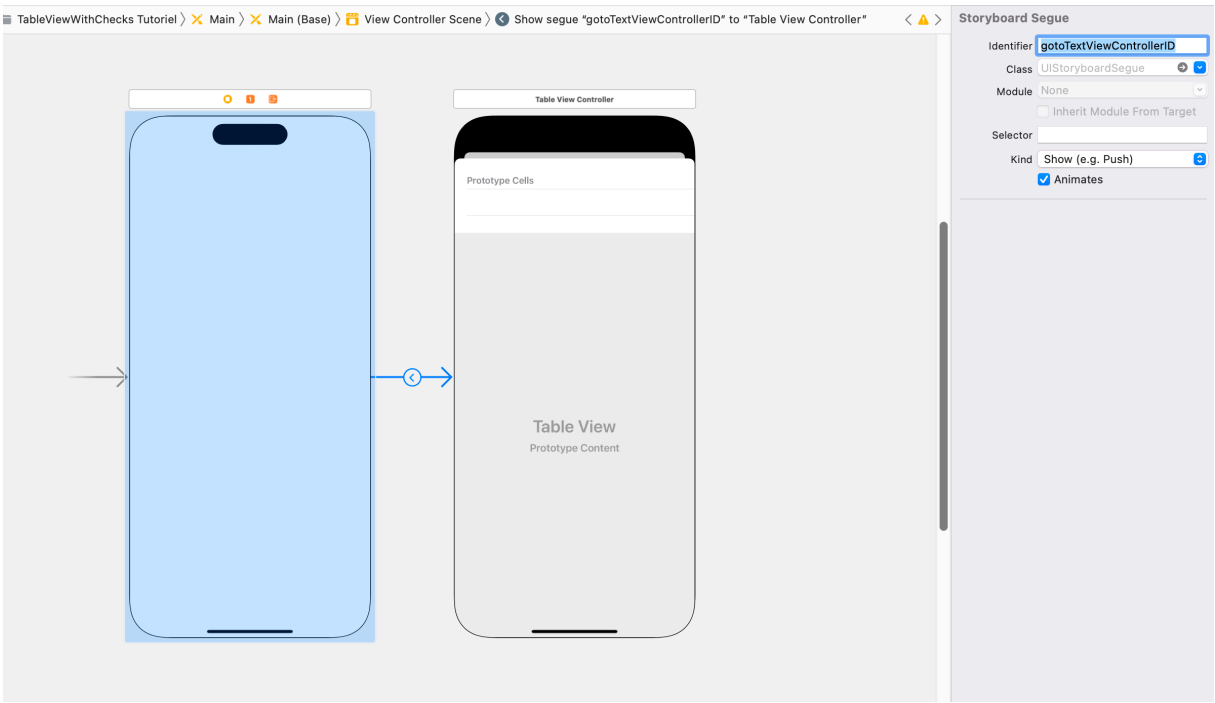


Figure 2Deux View Controllers le second étant un Table View Controller.

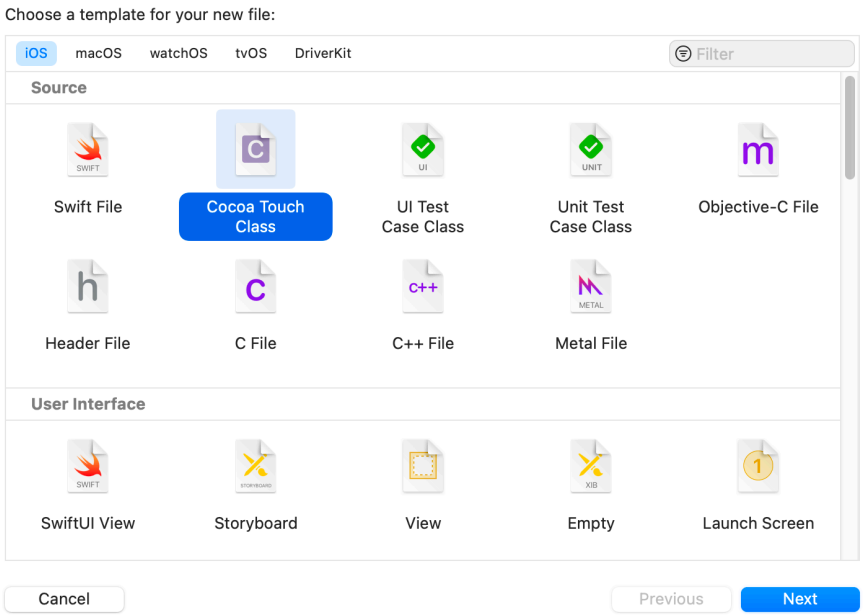
Nous allons maintenant créer un segue entre les deux. Pour cela cliquez sur le premier View Controller pour faire apparaître les trois icônes qui le surplombent ; sélectionnez le disque orange avec un carré blanc (il devient bleu), en maintenant la touche `control` glissez la souris jusqu'au deuxième View Controller en traçant un trait bleu ; le menu Segue apparait, sélectionnez Show.



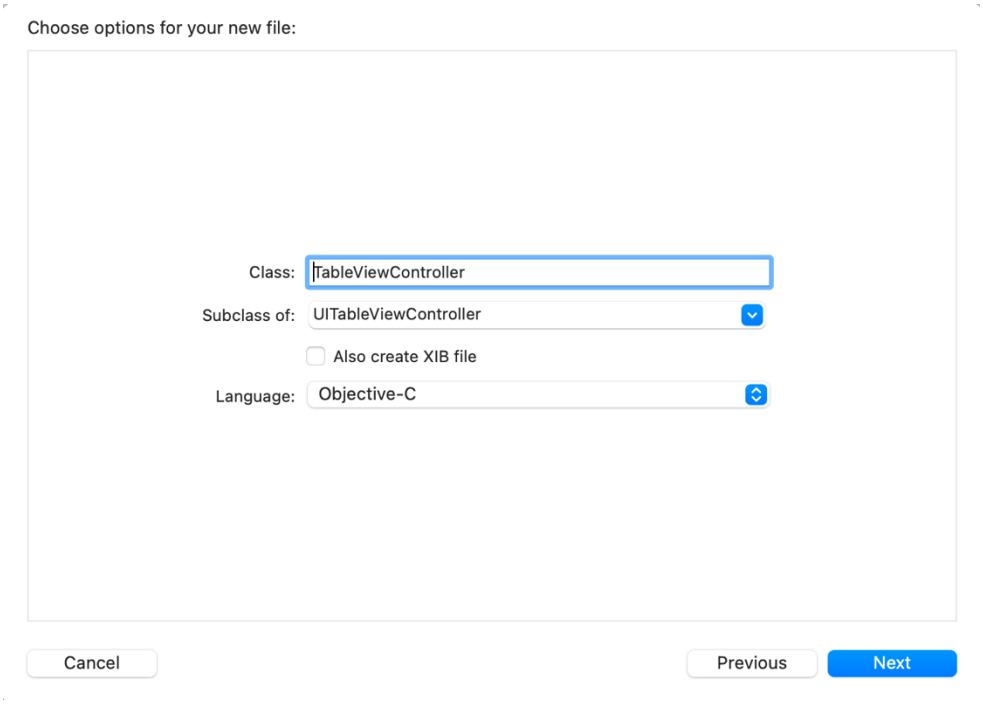
Nos deux View Controllers sont désormais liés par une transition segue. Sélectionnez cette flèche entre les deux View Controllers et dans l'`Attribute Inspector` donnez lui un identifiant (je propose `gotoTableViewID`).



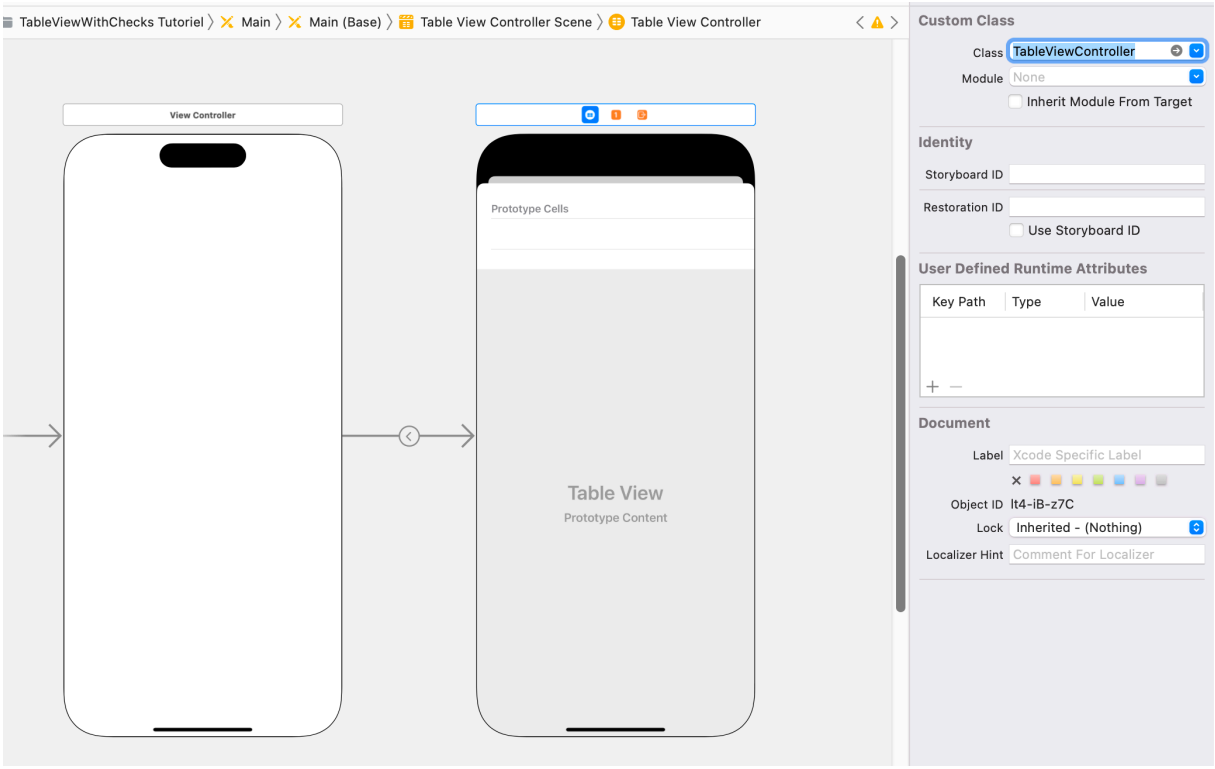
Comme précédemment, nous n'avons pas de classe pour contrôler le Text View Controller. Il nous faut l'ajouter : `File > New > File…` et choisissez le modèle `Cocoa Touch Class`.



Appelons-là `TableViewController` et faisons en sorte qu'elle hérite de `UITableViewController`, puis on l'ajoute dans le projet.



On retourne dans le `Main.storyboard`, on sélectionne notre Table View Controller, puis dans l'`Identifier inspector` du panneau latéral droit, on change le nom de la classe par `TableViewController`.



Sélectionnez ensuite la cellule et dans `Attribute Inspector` (toujours à droite), dans le champ Identifier mettez l'indentifiant `celluleID`.

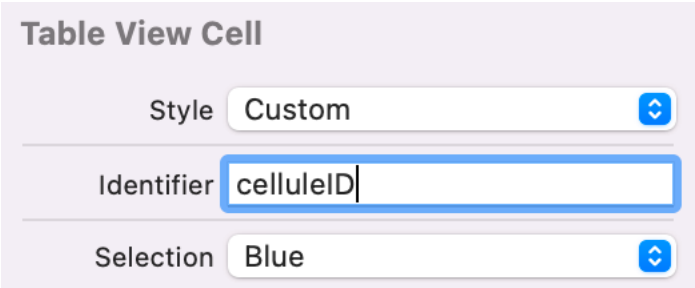
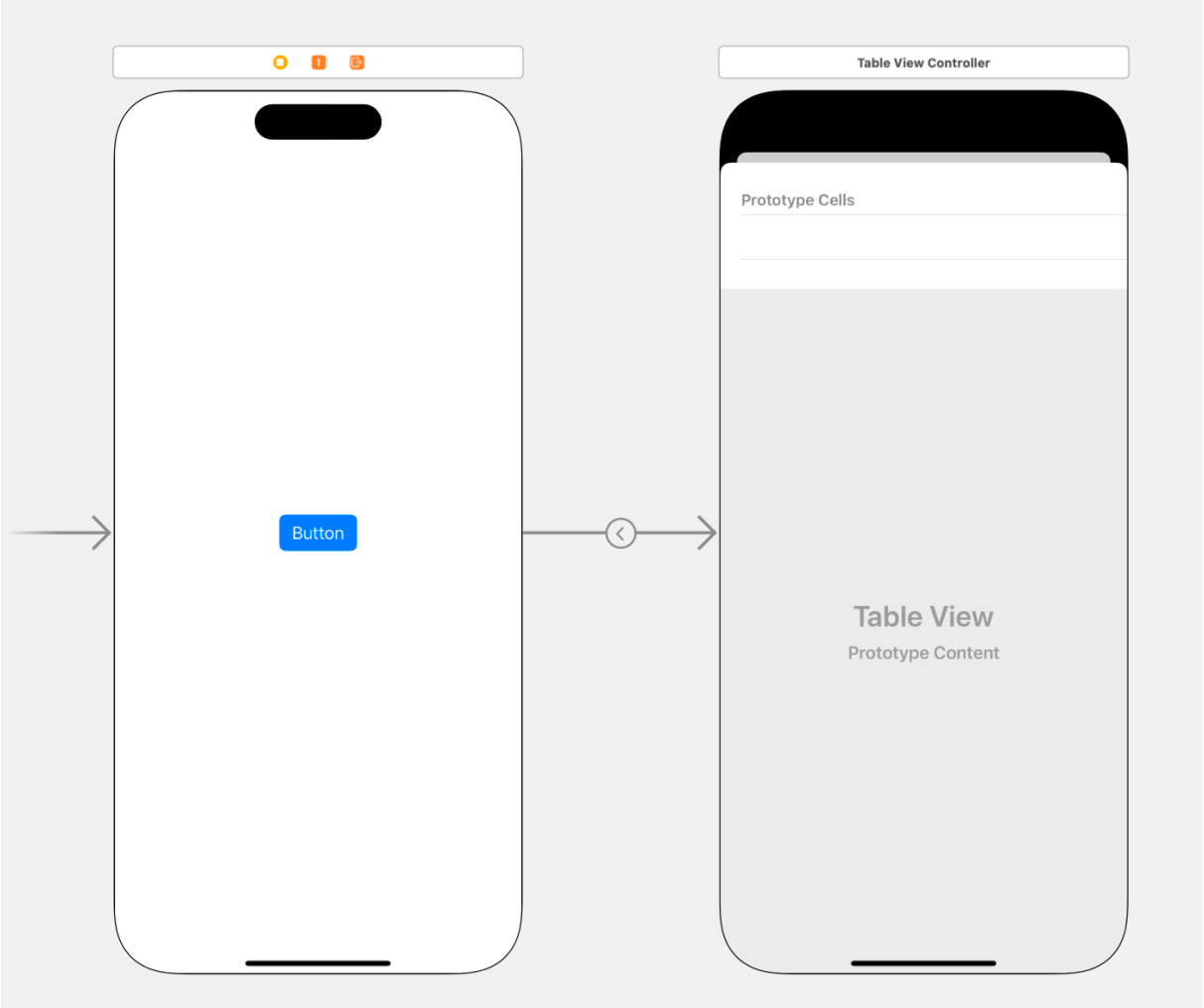
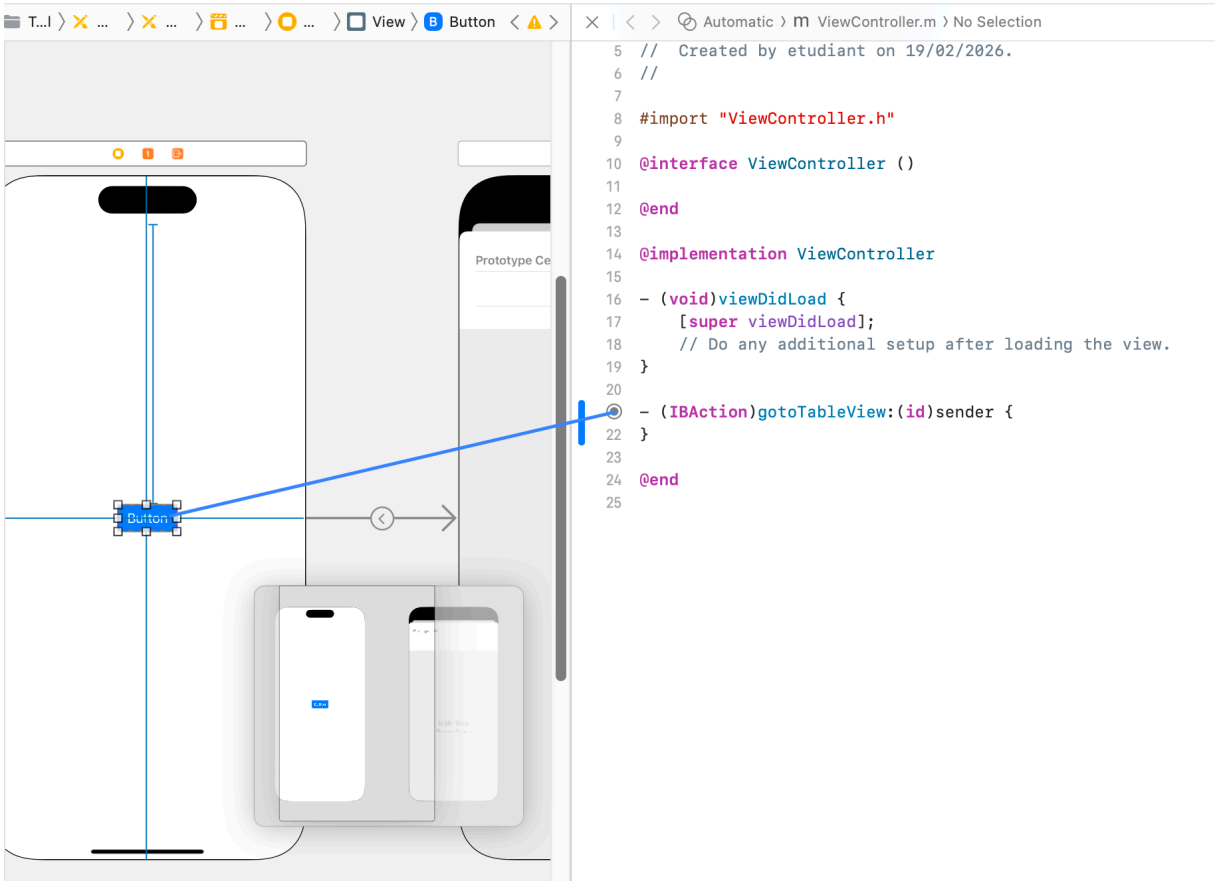


Figure 3On donne un identifiant à la cellule.

Afin de déclencher le passage du premier View Controller au Table View Controller, nous allons poser un bouton sur l'écran de gauche. On procède comme précédemment : on glisse le bouton depuis la bibliothèque de *widgets*.



Puis on passe en mode Assistant, on vérifie bien que dans la partie droite de l'écran on a le code de `ViewController.m`, on clique sur le bouton et en maintenant la touche control appuyée on glisse le pointeur de la souris pour ajouter une action (une méthode donc) qu'on appelle ici `gotoTableView`.



Codons

Ajoutez le code suivant à la méthode `gotoTableView:` pour qu'elle devienne :

```
- (IBAction)gotoTableView:(id)sender {
    [self performSegueWithIdentifier:@"gotoTableViewID" sender:self];
}
```

Comme précédemment, nous allons faire transiter une information du premier écran au second. Ici ce sera un `NSArray` `etudiants` mais ça pourrait, bien entendu, être n'importe quoi, comme un `int` pour représenter un socket par exemple.

Le `ViewController.h` devient :

```
//
//  ViewController.h
//  TableViewWithChecks Tuto
//

#import <UIKit/UIKit.h>

@interface ViewController : UIViewController ;
```

```
@property (readonly) NSArray* etudiants;

@end
```

De même on va ajouter deux propriétés (c'est vrai qu'on aurait pu faire autrement, par exemple utiliser une classe ou une `struct`) à la classe `TableViewController`, l'une pour représenter le tableau d'étudiants qu'on a reçu du `ViewController` précédent, et l'autre pour représenter le tableau des présents. Ce qui donne pour `TableViewController.h` :

```
//
//  TableViewController.h
//  TableViewWithChecks Tuto
//

#import <UIKit/UIKit.h>

NS_ASSUME_NONNULL_BEGIN

@interface TableViewController : UITableViewController
@property (copy, nonatomic) NSArray* receivedEtudiants;
@property (strong, nonatomic) NSMutableArray* presents;
@end

NS_ASSUME_NONNULL_END
```

On prend ensuite le code de Navigation pour préparer la transition avec depuis le segue `TableViewController.m` pour le mettre dans le `TableViewController.m` et on passe l'information au Table View Controller :

```
//
//  ViewController.m
//  TableViewWithChecks Tuto
//

#import "ViewController.h"
#import "TableViewController.h"

@interface ViewController ()

@end

@implementation OrangeViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.
}
```

```
- (IBAction)gotoTableView:(id)sender {
    [self performSegueWithIdentifier:@"gotoTableViewID"
                                sender:self];
}

#pragma mark - Navigation

// In a storyboard-based application, you will often want to do a
little preparation before navigation
- (void)prepareForSegue:(UIStoryboardSegue *)segue sender:(id)sender
{
    if ([segue.identifier isEqualToString:@"gotoTableViewID"]) {
      TableViewController *tableViewController ;
      tableViewController = [segue destinationViewController];
      tableViewController.receivedEtudiants = _etudiants;
    }
}

@end
```

Il nous suffit maintenant d'aller dans le `TableViewController.m`. Dans la méthode `viewDidLoad` nous allons initialiser le tableau des présents (on appelle son contructeur `initWithCapacity:`) puis on met toutes les valeurs de ce tableau à vrai (YES), ce qui donne pour cette méthode :

```
- (void)viewDidLoad {
    [super viewDidLoad];
    // on construit le tableau _presents
    self.presents = [[NSMutableArray alloc]
                     initWithCapacity:self.receivedEtudiants.count];
    // on mets toutes ses cases à VRAI
    for(int i=0; i<self.receivedEtudiants.count;i++)
        [self.presents addObject:@(YES)];
}
```

Il faut ensuite implémenter les deux méthodes indispensables à l'utilisation du protocole `TableView dataSource`. Celle qui donne le nombre de lignes dans le `Table View` :

```
- (NSInteger)tableView:(UITableView *)tableView
numberOfRowsInSection:(NSInteger)section {
    return self.receivedEtudiants.count;
}
```

Et celle qui explique quoi visualiser dans une cellule. Ici nous allons procéder par étape : on donne comme texte de la cellule, le nom de l'étudiant dans le tableau _receivedEtudiants correspondant à l'indice égal au numéro de la cellule ; on crée un UISwitch identifié (tag) par ce même numéro de cellule ; on donne à ce switch la valeur correspondante dans le tableau _presents ; on assigne une méthode qui sera appelée en

cas de changement de la valeur du switch ; on ajoute ce switch comme « accessoire » (accessoryView) de la cellule ; et enfin on retourne la cellule. Ce qui donnerait :

```
- (UITableViewCell *) tableView:(UITableView *)tableView
                      cellForRowAtIndexPath:(NSIndexPath *)indexPath
{
    UITableViewCell *cell =
           [tableView dequeueReusableCellWithIdentifier:@"celluleID"
                      forIndexPath:indexPath];
    // on ajoute le nom de l'étudiant à la cellule
    cell.textLabel.text = self.receivedEtudiants[indexPath.row];
    // on crée un switch
    UISwitch* switchControl = [[UISwitch alloc] init];
    // on assigne à ce switch le "tag" correspondant au numéro de
    // cellule (et donc ième étudiant)
    switchControl.tag = indexPath.row;
    // on place la valeur du switch à la valeur de présents
    // correspondante :
    switchControl.on = [self.presents[indexPath.row] boolValue];
    // on assigne une méthode qui sera appelée en cas de
    // changement de la valeur du switch (ici switchChanged:) :
    [switchControl addTarget:self action:@selector(switchChanged:)
                   forControlEvents:(UIControlEventValueChanged)];
    // on ajoute ce switch à la cellule :
    cell.accessoryView = switchControl;
    // on retourne la cellule :
    return cell;
}

// la méthode appelée en cas d'action sur le switch :
- (void)switchChanged:(UISwitch*) sender{
    NSInteger index = sender.tag;
    self.presents[index] = @(sender.isOn);
    //Optionnel : un petit printf() pour montrer
    // qu'on fait quelquechose.
    printf("L'étudiant %s est %s\n",
      [self.receivedEtudiants[index] UTF8String],
      [self.presents[index] isEqual:@(YES)] ? "présent" : "absent");
}
```

On peut tester l'application ainsi et se rendre compte qu'elle est totalement fonctionnelle. Néanmoins une IA bien intentionnée (et surtout bien utilisée), nous fait la remarque suivante : dans la méthode `tableView:cellForRowAtIndexPath:` nous avons utilisé `dequeueReusableCellWithIdentifier:`. Or cette méthode est très pratique pour économiser de la mémoire car iOS ne va pas créer 1000 cellules si on a 1000 étudiants mais seulement une dizaine (celles qui sont visibles à l'écran), et l'onsqu'on fait défiler la liste : les cellules qui sortent de l'écran sont recyclées pour afficher celles qui apparaissent. Comme à chaque fois on alloue un tout nouvel UISwitch en mémoire (`[[UISwitch alloc] init];`) et qu'on l'ajoute à la cellule courante avec `accessoryView` on écrase l'ancien. Donc l'interface passe son temps à détruire et

recréer des éléments en boucle, ce qui consomme en vain du processeur et de la mémoire ce qui pourrait éventuellement saccader le comportement de l'application en cas de très grandes promotions d'étudiants. La solution proposée pour corriger ce « bug » est de vérifier si la cellule à afficher possède déjà un switch. Si c'est le cas, on le réutilise, sinon on le crée et on l'attache à la cellule.

Finalement le code de `TableViewController.m` devient :

```
//
//  TableViewController.m
//  TableViewWithChecks Tuto
//

#import "TableViewController.h"

@interface TableViewController ()

@end

@implementation TableViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    self.presents = [[NSMutableArray
alloc]initWithCapacity:self.receivedEtudiants.count];
    for(int i=0; i<self.receivedEtudiants.count;i++)
        [self.presents addObject:@(YES)];
}

#pragma mark - Table view data source

- (NSInteger)tableView:(UITableView *)tableView
numberOfRowsInSection:(NSInteger)section {
    return self.receivedEtudiants.count;
}


- (UITableViewCell *)tableView:(UITableView *)tableView
                     cellForRowAtIndexPath:(NSIndexPath *)indexPath
{
    UITableViewCell *cell =
           [tableView dequeueReusableCellWithIdentifier:@"celluleID"
                       forIndexPath:indexPath];

    // 1. On met à jour le texte
    cell.textLabel.text = self.receivedEtudiants[indexPath.row];

    UISwitch *switchControl;
```

```
    // 2. On vérifie si la cellule possède déjà un UISwitch
    if ([cell.accessoryView isKindOfClass:[UISwitch class]]) {
        // La cellule a été recyclée et possède déjà un switch,
        // on le récupère
        switchControl = (UISwitch *)cell.accessoryView;
    } else {
        // C'est une nouvelle cellule, on crée le switch de zéro
        switchControl = [[UISwitch alloc] init];

        // On n'ajoute la méthode (Target) qu'une seule fois
        // lors de la création !
        [switchControl addTarget:self
                        action:@selector(switchChanged:)
                        forControlEvents:UIControlEventValueChanged];

        cell.accessoryView = switchControl;
    }

    // 3. Dans TOUS les cas, on met à jour les données du switch
    // pour cette ligne précise
    switchControl.tag = indexPath.row;
    switchControl.on = [self.presents[indexPath.row] boolValue];

    return cell;
}

// la méthode appelée en cas d'action sur le switch :
- (void)switchChanged:(UISwitch*) sender{
    NSInteger index = sender.tag;
    self.presents[index] = @(sender.isOn);
    //Optionnel : un petit printf() pour montrer qu'on fait
quelquechose.
    printf("L'étudiant %s est %s\n",
      [self.receivedEtudiants[index] UTF8String],
      [self.presents[index] isEqual:@(YES)] ? "présent" : "absent");
}

@end
```

On peut bien évidemment constater en exécutant l'application qu'elle est toujours totalement fonctionnelle (*cf* figure 4).

Néamoins quand on regarde l'affichage dans la console de log associée à l'application, on a :

```
2026-02-20 15:35:16.049531+0100 TableViewWithChecks
Tutoriel[11624:282877] invalid mode 'kCFRunLoopCommonModes' provided
to CFRunLoopRunSpecific - break on
```

```
_CFRunLoopError_RunCalledWithInvalidMode to debug. This message will
only appear once per execution.
L'étudiant Pierre est absent
L'étudiant Jacques est absent
```

Cette première ligne est inquiétante. Elle apparait lorsqu'on agit pour la première fois sur un des switchs. Elle est due, en fait, à un bug d'affichage très connu du simulateur qui est apparu avec Xcode 14. Il n'y a rien de grave. C'est totalement sans conséquence (même si ça fait tâche).

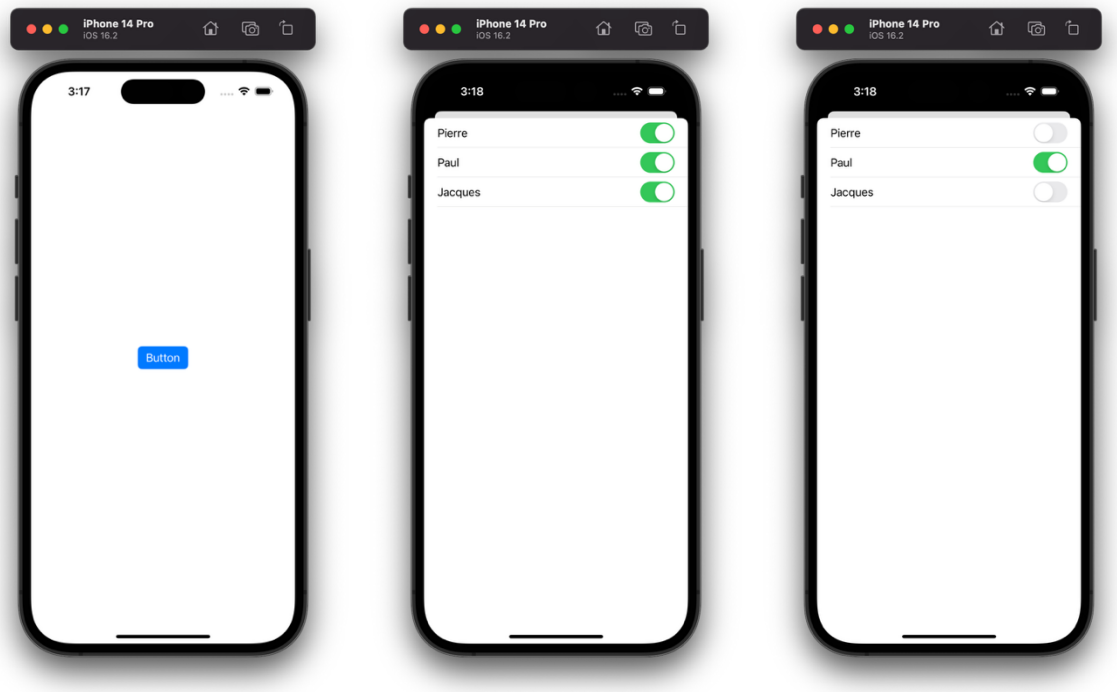


Figure 4 L'application fonctionne.

L'inconvénient de notre application est qu'elle ne conserve pas l'état des présents quand on repasse au `ViewController` initial. Si l'on veut faire ça, il est nécessaire que l'état des présents soit conservé dans ce dernier (comme le tableau des étudiants). Pour cela, c'est assez simple, il suffit de déclarer une nouvelle propriété dans le `ViewController.m` (pas dans le `.h` car elle n'a pas besoin d'être publique). Dans ce même fichier on va faire l'initialisation de ce tableau (on le faisait dans le `viewDidLoad` du `TableViewController` on la déplace dans le `viewDidLoad` du `ViewController`. Puis on passe sa valeur dans le `prepareForSegue:sender:`. Ce qui donne :

```
//
//  ViewController.m
//  TableViewWithChecks Tutoriel
//

#import "ViewController.h"
```

```
#import "TableViewController.h"

@interface ViewController ()
@property (strong, nonatomic) NSMutableArray* presentsState;
@end

@implementation ViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    _etudiants = @[@"Pierre",@"Paul",@"Jacques"];

    // C'est maintenant ici qu'on initialise le tableau des présents
    _presentsState =
       [[NSMutableArray alloc] initWithCapacity:_etudiants.count];
    for (int i=0; i<_etudiants.count; i++)
        [self.presentsState addObject:@(YES)];
}

- (IBAction)gotoTableView:(id)sender {
    [self performSegueWithIdentifier:@"gotoTableViewID"
                                sender:self];
}


#pragma mark - Navigation
- (void)prepareForSegue:(UIStoryboardSegue *)segue sender:(id)sender
{
    if ([segue.identifier isEqualToString:@"gotoTableViewID"]) {
        TableViewController* tableViewController;
        tableViewController = segue.destinationViewController;
        tableViewController.receivedEtudiants = _etudiants;
        tableViewController.presents = _presentsState;
    }
}
@end
```